1.1.6. Dijkstra's S... 12:05

Given a graph G and source vertex S, Dijkstra's shortest path algorithm is used to find the shortest paths from source S to all vertices in the given graph.

The Dijkstra algorithm is also known as the single-source shortest path algorithm. It is based on the greedy technique. A little variation in the algorithm can find the shortest path from the source nodes to all the other nodes in the graph.

The function **void dijkstra(int** $G[MAX][MAX]$**, int** n**, int** $startnode$**)** computes and prints the shortest path distances and corresponding paths from the given source node to all other nodes in a weighted directed graph using Dijkstra's algorithm. It outputs the distance or "INF" if unreachable, along with the path or "NO PATH" for each node.

Note:

- Vertices are numbered from 1 through V.
- All input values are separated by spaces and/or newlines.

Sample Input and Output:

```
Enter the number of vertices : 4
Enter the number of edges : 5
Enter source : 1
Enter destination : 2
Enter weight : 4
Enter source : 1
Enter destination : 4
Enter weight : 10
Enter source : 1
Enter destination : 3
Enter weight : 6
Enter source : 2
Enter destination : 4
```

Explorer

Dijkstras.c

Submit

Debugger

```c
#include <limits.h>
#include <stdio.h>
#define MAX 20
int V, E;
int graph[MAX][MAX];
#define INFINITY 99999
//Name: SAATWIK SINHA
//PRN:25070521077
void dijkstra(int G[MAX][MAX], int n, int startnode) {
    int cost[MAX][MAX], distance[MAX],pred[MAX];
    int visited[MAX],count,mindistance, nextnode, i, j;

    for(i=1; i<=n; i++){
        for(j=1; j<=n; j++){
            if(G[i][j] == 0)
                cost[i][j] = INFINITY;
            else
                cost[i][j] = G[i][j];
        }
    }

    for(i = 1; i <= n; i++){

distance[i]=cost[startnode][i];
        pred[i] = startnode;
```

‹ Prev Reset Submit Next ›

```
Enter source : 3
Enter destination : 4
Enter weight : 2
Enter the source :1
Node    Distance    Path
  2     4   2<-1
  3     6   3<-1
  4     8   4<-3<-1
```

Sample Test Cases +

```
    }


distance[startnode] = 0;

visited[startnode]=1;
    pred[startnode]=-1;
    count=1;

    while(count < n-1){
        mindistance = INFINITY;

        for(i=1; i<=n; i++){

if(distance[i] <mindistance && !visited[i]){

mindistance = distance[i];

nextnode=i;
            }
        }

visited[nextnode]=1;

        for(i=1; i<=n; i++){

if(!visited[i]){

if(mindistance + cost[nextnode][i] < distance[i]){

distance[i]=mindistance + cost[nextnode][i];

pred[i]=nextnode;
```

```
            }
        }
        count++;
    }


printf("Node\tDistance\tPath\n");
    for(i=1; i<=n; i++){
        if(i != startnode){
if(distance[i]==INFINITY){
printf("   %d\tINF\tNO PATH\n", i);
            }
            else{
printf("   %d\t%d\t",i,distance[i]);
                int crawl = i;
printf("%d", crawl);
while(pred[crawl] != -1){
printf("<-%d", pred[crawl]);
crawl = pred[crawl];
                }
printf("\n");
            }

        }
    }
```

```
76  int main() {
77      int s, d, w, i, j;
78      printf("Enter the
        number of vertices :
        ");
79      scanf("%d", &V);
80      printf("Enter the
        number of edges : ");
81      scanf("%d", &E);
82      for(i = 1 ; i <=
        V; i++) {
83          for(j = 1; j
        <= V; j++) {
84              graph[i]
        [i] = 0;
85          }
86      }
87      for(i = 1; i <= E;
        i++) {
88          printf("Enter
        source : ");
89          scanf("%d",
        &s);
90          printf("Enter
        destination : ");
91          scanf("%d",
        &d);
92          printf("Enter
        weight : ");
93          scanf("%d",
        &w);
94          if(s > V || d
        > V || s <= 0 || d <=
        0) {
95
        printf("Invalid index.
        Try again.\n");
96              i--;
97              continue;
98          } else {
99              graph[s]
        [d] = w;
100         }
```